BRAUDE
College of Engineering, karmiel

Software Engineering Department

Braude College of Engineering

Capstone Project – Phase B (61999)

Evolution of Knowledge Areas in Publications

CODE: 26-1-R-16

By:

Yamit Moreno yamitmoreno1@gmail.com

Shaked Tadessa shakedt67509@gmail.com

Advisors:

Dr. Dvora Toledano-Kitai

Prof. Zeev Volkovich

Link to GitHub:

https://github.com/Yamit-Moreno/FinalProject_EvolvingKnowledgeInPublications

Abstract

This project presents a framework for analyzing structural changes in large-scale dynamic networks, using academic citation networks as a case study. It identifies patterns, trends, and transformations in research communities over time, while distinguishing meaningful structural changes from noise in complex networks.

The approach integrates GraphSAGE-based node embeddings with K-Means clustering to group similar nodes and evaluates cluster quality using the Silhouette Score. Temporal cluster dynamics, including merges, splits, and emerging research domains, are visualized using Alluvial diagrams to represent community evolution.

Overall, the framework enables robust identification of evolving network structures under noisy conditions and supports dynamic network analysis. Its main contribution is a structured view of network evolution, with applicability to domains such as social, transportation, and biological networks.

1. Introduction & Problem Formulation

Citation network analysis is important for understanding how knowledge and ideas evolve over time. Existing analytical tools can reveal patterns and structural relationships within these networks, but they often fail to capture dynamic evolution of research communities and domains over time. Understanding these changes is essential for identifying emerging trends, as well as splits and merges of communities, and the formation of new domains.

Large weighted networks are commonly used to represent complex systems such as scientific citation networks, and they typically exhibit modular structures in the form of communities or clusters. However, when these structures evolve over time, it becomes difficult to distinguish between genuine structural changes and variations caused by random fluctuations in edge weights. The central challenge is to identify which observed changes in network structure reflect true evolution of research communities, and which are the result of noise or instability in the data.

This challenge appears in two main aspects: (1) edge-weight noise, where random fluctuations in connection strengths may create or hide apparent clusters without reflecting true structural changes, and (2) the complexity of representing and interpreting change over time, where even real structural changes are difficult to clearly describe and visualize over time.

The goal of this project is to analyze structural changes in citation networks and track the evolution of research communities over time, and to study how groups of papers form communities, how research domains evolve, and when new domains emerge or existing ones shift.

The approach is based on representing each paper as a node embedding using GraphSAGE, capturing both local and global network structure. K-Means clustering is then used to group similar papers, and cluster quality is evaluated using the Silhouette Score to ensure a meaningful representation of the underlying structure. Finally, temporal changes in clusters are visualized using Alluvial Diagrams, allowing an intuitive representation of community evolution over time.

2. Theoretical Background and Preliminaries

This project belongs to the field of complex networks [7] and the study of knowledge and information dynamics. Its aim is to analyze dynamic citation networks; to better understand how ideas, interests, and scientific knowledge evolve over time.

The project intersects several scientific areas. In Information Science, it focuses on the analysis of network structure, identification of communities, and detection of relationships between papers, enabling a deeper understanding of knowledge organization. In Machine Learning and Data Science, it applies numerical node representations (embeddings) and algorithmic methods for analyzing and clustering network structure. In Social Sciences and interdisciplinary studies, it supports the analysis of knowledge dissemination and the identification of emerging scientific trends and research domains.

Overall, the project focuses on the dynamics of complex information systems, combining approaches from computer science, statistics, and social sciences to provide a consistent view of how scientific knowledge evolves.

2.1 Citation Networks, Embeddings and Clustering

A network is a mathematical structure consisting of nodes and edges representing relationships between them. Many real-world networks are dynamic, meaning they evolve over time through the addition or removal of nodes and edges. This allows the study of how structural properties change and how knowledge domains develop in a temporal context.

A citation network [8] is a directed graph where nodes represent scientific papers and edges represent citation relationships. These networks capture the flow of scientific information and allow the analysis of influence and connectivity between research areas.

Due to their large scale and high connectivity, citation networks are considered complex networks, in which communities and research domains continuously emerge and evolve. Communities represent research topics, and tracking their evolution over time enables the identification of stable domains, emerging fields, and structural changes such as splits and merges.

In citation network analysis, each node is represented by a vector embedded in a high-dimensional space. These embeddings capture both local neighborhood structure and global network relationships, enabling machine learning-based analysis of graph structure.

Embedding methods transform nodes into numerical representations that enable computational analysis. Classical approaches rely mainly on local neighborhood structure, while modern graph learning methods incorporate both local and global information.

GraphSAGE [1] is a representative method that learns inductive node embeddings by aggregating information from sampled neighbors. This allows scalable representation learning for large and dynamic networks and improves the ability to capture structural and semantic properties of citation graphs.

At each layer, the algorithm performs two main steps: sampling a fixed-size set of neighbors and aggregating their features using a learnable function (e.g., mean or pooling). By stacking multiple layers, the model captures higher-order neighborhood information, enabling embeddings that reflect both local structure and broader graph context.

The final output is a compact vector representation for each node, used for downstream tasks such as clustering and community detection.

Primary clustering is performed using K-Means [5], which groups nodes with similar embeddings into clusters representing research communities. The algorithm iteratively assigns points to the nearest centroid and updates centroids as the mean of assigned points until convergence.

Despite not directly using graph structure, K-Means is widely used due to its efficiency and scalability. Since real-world networks contain noise and structural variability, clustering results may vary, making evaluation necessary.

Cluster quality is assessed using the Silhouette Score [10], which measures how well each node fits within its assigned cluster compared to other clusters. Higher values indicate more meaningful and well-separated clusters, while lower values indicate overlap or ambiguous assignments. The score is used to compare clustering configurations and select the optimal number of clusters.

Jaccard Similarity measures the overlap between two sets as the ratio between their intersection and union. It is used to compare cluster memberships and evaluate similarity between cluster assignments over time or across runs. The score ranges between 0 and 1, where higher values indicate stronger similarity between sets.

The Hungarian algorithm solves the assignment problem by finding an optimal one-to-one matching between two sets based on a cost matrix. It is used in this project to align clusters across different time snapshots by minimizing dissimilarity.

Alluvial Diagrams are used to visualize changes in cluster structure over time. They represent clusters as blocks at different time steps and transitions between them as flows, where flow width reflects the number of nodes. This provides an intuitive representation of community evolution, including persistence, splits, merges, and emergence of new research domains.

2.2 Preliminaries

Graph Representation

A citation network is represented as a directed graph: $G = (V, E)$

where V is the set of nodes (papers) and $E \subseteq V \times V$ is the set of directed citation edges.

Temporal Snapshots

The dynamic graph is divided into a sequence of temporal snapshots: $G_1, G_2, ..., G_T$

where each G_t represents the subgraph corresponding to a specific time window.

A cumulative snapshot may also be defined as: $G_t = (V_{\leq t}, E_{\leq t})$

Node Embeddings

Each node $v_i \in V$ is represented as a vector in a d-dimensional embedding space: $x_i \in \mathbb{R}^d$

where the embedding encodes both structural and semantic information.

Clustering Definition

Given a set of embeddings $X = \{x_1, ..., x_n\}$, clustering aims to partition the nodes into K disjoint sets: $C_1, C_2, ..., C_K$

such that:

$$\bigcup_{k=1}^{K} C_k = V, \quad C_i \cap C_j = \emptyset$$

Similarity Between Clusters

Cluster similarity is defined using **Jaccard similarity**:

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

Temporal Alignment

Given two sets of clusters at consecutive time steps, alignment is formulated as an assignment problem solved using the **Hungarian algorithm** by maximizing similarity.

2.3 Model Flowchart

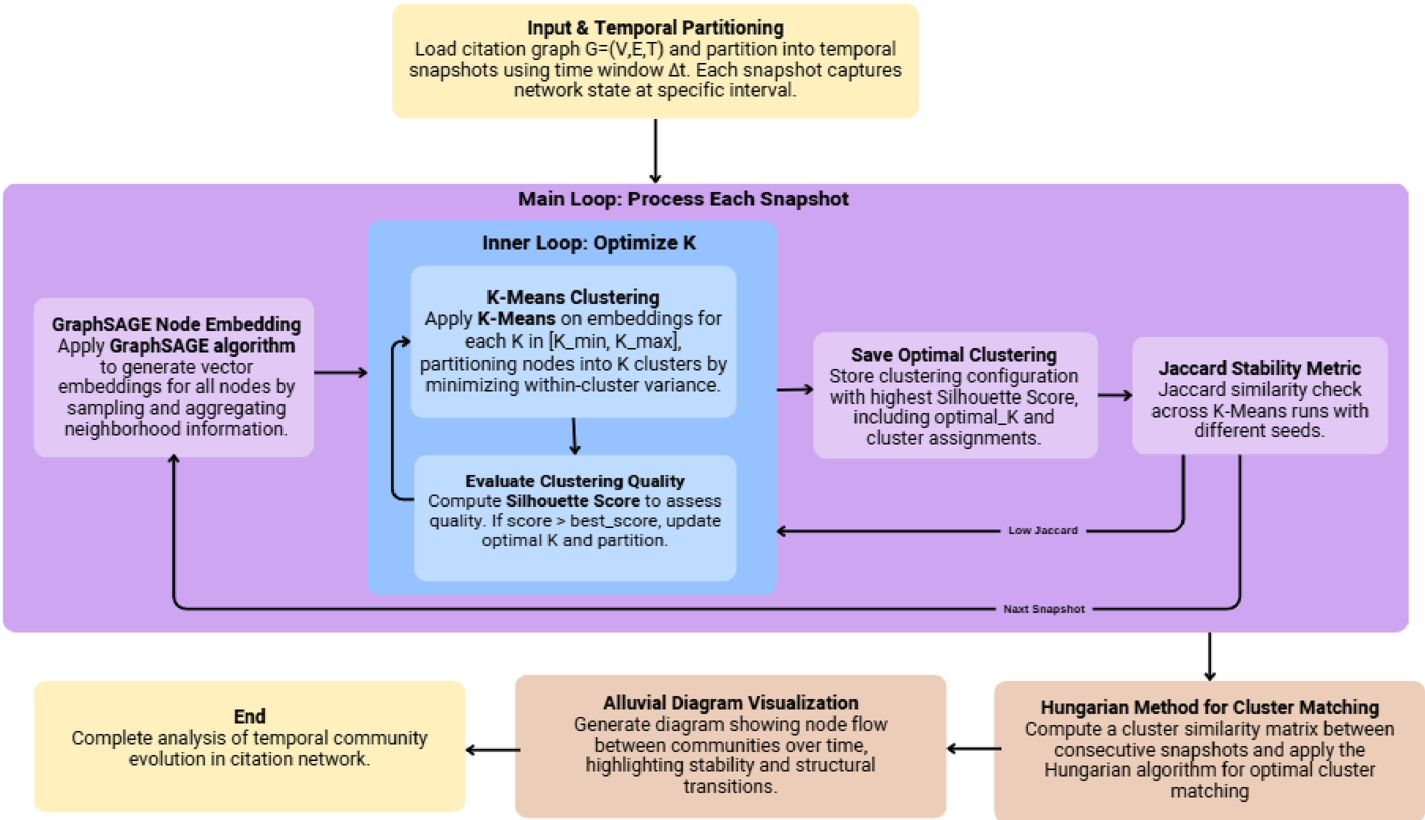

Figure 1: Dynamic citation network analysis algorithm

3. Research Process and System Implementation

This section presents the proposed approach, system design, and implementation details of the project. The goal is to analyze temporal evolution in citation networks by combining graph representation learning, clustering methods, and temporal alignment techniques.

3.1 Proposed Approach

The proposed approach is designed to detect and analyze evolving research communities in dynamic citation networks. The system transforms raw citation data into structured temporal graphs, learns node representations, clusters similar papers into communities, and tracks the temporal evolution of these communities. The pipeline consists of the following main stages:

- Data preprocessing and graph construction
- Temporal snapshot generation
- Feature engineering
- Graph representation learning using GraphSAGE
- Community detection using K-Means clustering
- Optimal selection of number of clusters
- Temporal alignment of communities
- Visualization of community evolution

Each stage operates per temporal snapshot, followed by cross-snapshot comparison and alignment to capture temporal dynamics.

3.1.1 Stage 0: Data Processing and Graph Construction

In this stage, raw citation data is loaded and transformed into a structured graph.

The dataset includes:

- Scientific papers (nodes)
- Citation relationships (edges)
- Metadata such as publication year and field of study

Data cleaning is performed to ensure graph validity by removing:

- Self-citations
- Invalid or missing references
- Isolated or disconnected nodes (when appropriate for analysis)

The resulting structure is a directed graph: $G = (V, E)$

where nodes represent papers and edges represent citation relationships.

3.1.2 Stage 1: Temporal Snapshot Construction

The graph is divided into multiple temporal snapshots based on publication years.

Each snapshot represents a subgraph: $G_t = (V_t, E_t)$

Two types of snapshots are used:

- **Time-window snapshots** (fixed intervals)
- **Cumulative snapshots** (all data up to time t)

This enables analysis of both short-term and long-term structural changes in the network while preserving temporal consistency.

3.1.3 Stage 2: Feature Engineering

Each node (paper) is represented using a feature vector that captures structural and semantic properties of the graph.

The features include:

- Number of incoming citations (in-degree)
- Number of outgoing citations (out-degree)
- Encoded field of study information

These features form a node feature matrix used as input for the graph learning model in each temporal snapshot.

3.1.4 Stage 3: Graph Conversion (PyTorch Geometric Format)

The structured graph is converted into PyTorch Geometric (PyG) format to support graph neural network processing.

The representation includes:

- Node feature matrix
- Edge index (COO format)
- Mapping between node IDs and indices

This format enables efficient training of Graph Neural Networks on large-scale graphs.

3.1.5 Stage 4: Node Representation Learning (GraphSAGE)

Node embeddings are learned using a GraphSAGE-based model.

The model generates a vector representation for each node by aggregating information from its neighbors.

This process allows each paper to be represented in a low-dimensional embedding space that preserves both local and global graph structure.

The model is trained independently for each temporal snapshot in order to capture temporal structural changes.

3.1.6 Stage 5: Community Detection (K-Means Clustering)

After embeddings are generated, K-Means clustering is applied to group similar nodes into communities.

Each cluster represents a group of papers with similar structural and semantic properties, which can be interpreted as a research topic or domain.

3.1.7 Stage 6: Optimal Cluster Selection

Since the number of communities is not known in advance, multiple values of K are tested.

For each K, clustering quality is evaluated using the Silhouette Score.

The optimal number of clusters is selected based on the highest score:

$$K^* = \arg\max_K \mathrm{Silhouette}(K)$$

This ensures adaptive clustering for each snapshot.

3.1.8 Stage 7: Temporal Alignment of Communities

Since clustering is performed independently for each snapshot, cluster labels are not consistent across time.

To enable comparison across time, clusters are aligned using:

- Jaccard similarity to measure overlap between clusters
- Hungarian algorithm to compute optimal one-to-one matching

This alignment allows tracking:

- Persistent communities
- Emerging communities
- Splitting and merging structures
- Disappearing communities

3.1.9 Stage 8: Visualization of Community Evolution

Community evolution is visualized using Alluvial Diagrams.

These diagrams represent:

- Clusters as nodes at each time step
- Flows between clusters across time
- Flow thickness proportional to the number of shared nodes

This provides an intuitive view of how research communities evolve, split, merge, and emerge over time in a continuous temporal framework.

3.2 Research Process

The system is executed as an iterative experimental pipeline.

For each time window:

1. Load and construct graph snapshot
2. Build node features
3. Generate GraphSAGE embeddings
4. Apply K-Means clustering
5. Select optimal number of clusters
6. Evaluate clustering robustness
7. Store results for temporal comparison

This process is repeated over multiple time windows to analyze long-term temporal evolution patterns.

3.3 System Implementation: Codebase, Language & Libraries

The system is implemented in Python and structured into modular components to ensure maintainability and scalability.

Main modules:

- **data/** – data loading, cleaning, snapshot construction
- **embeddings/** – GraphSAGE model and embedding generation
- **clustering/** – K-Means optimization and robustness evaluation
- **alignment/** – cluster similarity and temporal matching
- **validation/** – evaluation metrics (Silhouette score, stability measures)
- **visualization/** – alluvial diagram generation

Key libraries used:

- PyTorch – deep learning framework
- PyTorch Geometric – graph neural networks
- Scikit-learn – clustering and evaluation metrics
- NumPy & Pandas – data processing
- SciPy – Hungarian matching algorithm
- Matplotlib / Plotly – visualization

The system is executed in a Google Colab environment with Google Drive integration for data storage and project organization.

3.4 Challenges During Implementation

Several challenges were encountered during system development:

1. Scalability of Large Graphs

Citation networks are large and sparse, requiring efficient processing strategies. This was addressed using snapshot-based computation and PyG optimization.

2. Noisy and Incomplete Data

Real-world citation data includes missing or inconsistent references, requiring extensive preprocessing and filtering.

3. Selecting Optimal Number of Clusters

Choosing the number of communities per snapshot is non-trivial and was solved using Silhouette-based evaluation.

4. Temporal Inconsistency of Clusters

Clusters are independently generated per snapshot, requiring alignment using similarity measures and the Hungarian algorithm.

5. Embedding Variability

GraphSAGE training introduces randomness between runs, requiring robustness evaluation across multiple executions.

6. Visualization Complexity

Representing multi-step temporal evolution in an interpretable way required the design of alluvial diagrams.

4. Results and Conclusions

4.1 Results and Evaluation

4.1.1 Experimental Setup

The proposed framework was evaluated on four cumulative citation network snapshots covering the periods:

2000–2005, 2000–2010, 2000–2015, and 2000–2020. For each snapshot, the full pipeline was applied, including GraphSAGE embedding generation, K-Means clustering, optimal cluster selection using the Silhouette Score, robustness evaluation, and temporal alignment between consecutive snapshots.

4.1.2 Quantitative Summary of Results

Time Window	Papers	Edges	Categories	Best K	Silhouette Score	Robustness Mean	Robustness Std
2000–2005	2,466	706	7	10	0.896997	0.931288	0.068712
2000–2010	21,983	27,519	8	10	0.898320	0.908003	0.071905
2000–2015	25,502	30,917	8	9	0.822343	0.798616	0.079437
2000–2020	25,562	31,003	8	10	0.801391	0.854567	0.054467

Table 1: Summary of results across time windows

Table.1 summarizes the key quantitative results for each time window.

The number of papers increases significantly from 2,466 to 25,562, indicating substantial network growth. Similarly, the number of edges increases from 706 to 31,003, reflecting a denser and more complex citation structure.

The number of FOS categories remains relatively stable (7–8), suggesting that growth is driven primarily by intra-field expansion rather than the emergence of new fields.

The optimal number of clusters (Best K) remains around 9–10 across snapshots, indicating that detected communities represent sub-structures within broader research domains rather than direct mappings to FOS categories. . Accordingly, clusters should be interpreted as sub-communities within broader domains (e.g., Computer Science), rather than as a one-to-one mapping to FOS categories.

4.1.3 Clustering Quality and Cluster Selection

Clustering quality was evaluated using the **Silhouette Score**.

Across all snapshots, values range from 0.791 to 0.894, indicating good separation and strong internal cohesion.

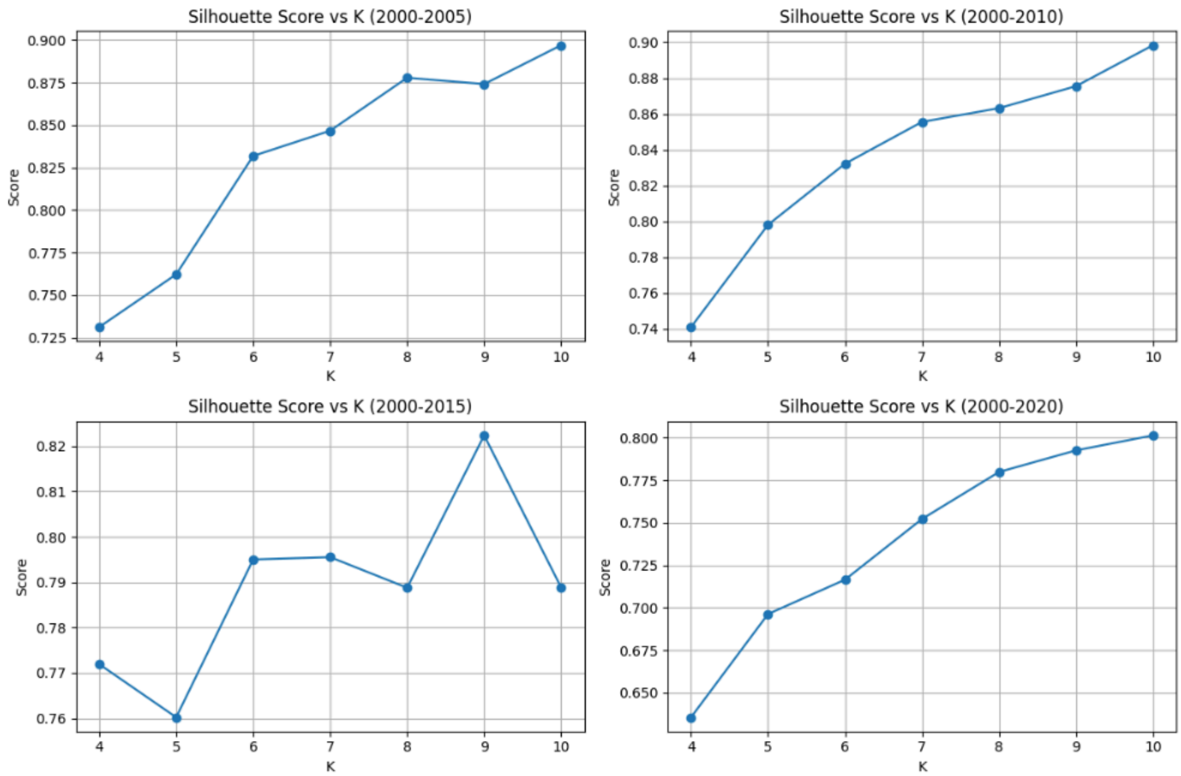

Figure 2: Silhouette Score vs K

The graphs illustrate the cluster selection process across time windows.
For the 2000–2005, 2000–2010, and 2000–2020 snapshots, the optimal value is K=10.
For the 2000–2015 snapshot, the optimal value is K=9.

This indicates that the number of communities is not fixed, but adapts to the underlying structure of each snapshot.

4.1.4 Clustering Robustness

Clustering robustness was evaluated across multiple K-Means runs with different random initializations.

High robustness values indicate consistent cluster structures and reliable results.
The slightly lower robustness observed in the 2000–2015 snapshot suggests reduced stability during that period, but remains within an acceptable range.

The low standard deviation further confirms consistency across runs.

4.1.5 Clustering Size Distribution

Cluster size distribution provides additional insight into the internal structure of the detected communities.

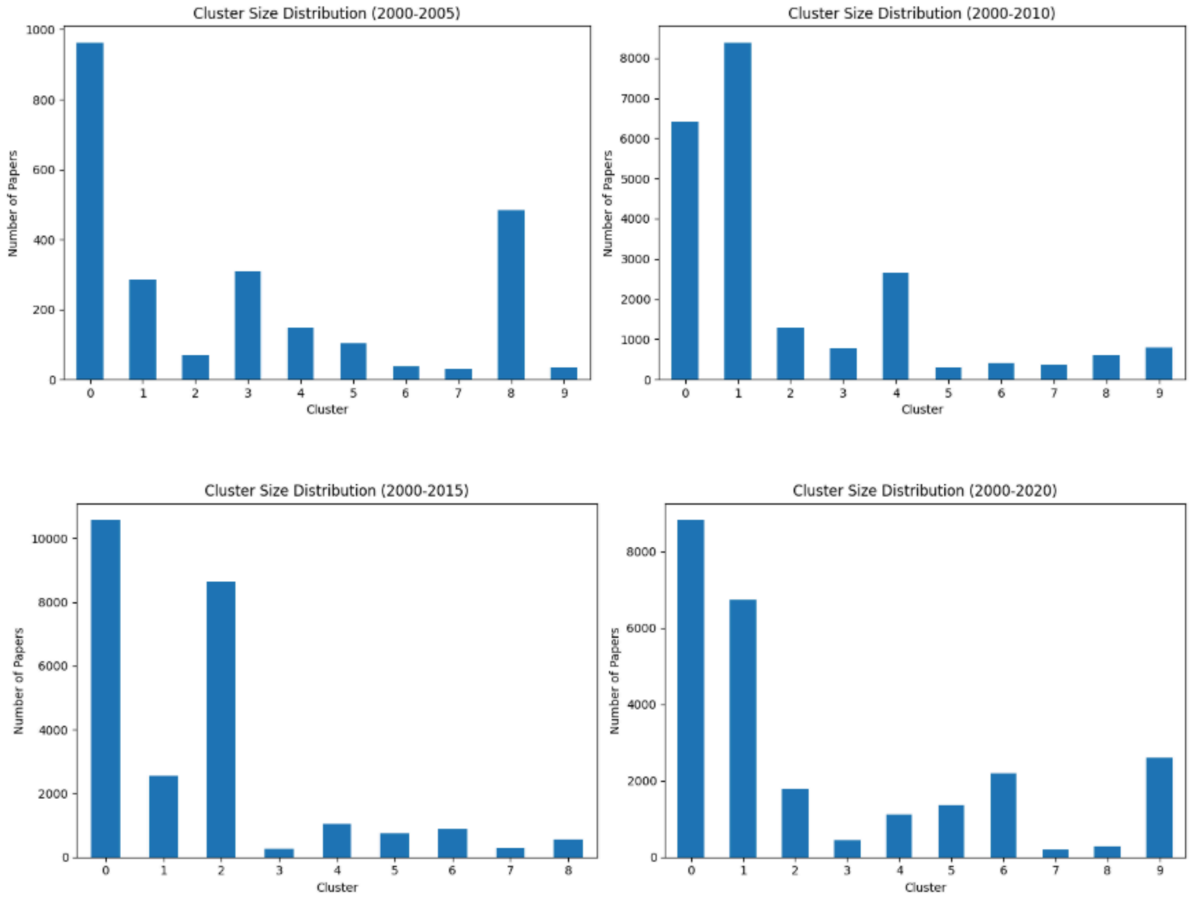

Figure 3: Cluster Size Distribution

Fig. 2 shows that the clusters are not equal in size: in each time window, a small number of large clusters contain a major portion of the papers, while several smaller clusters represent more focused or specialized communities.

This imbalance becomes more pronounced as the network grows over time.

For example, in the 2000–2020 snapshot, the two largest clusters contain 8,817 and 6,747 papers, while several smaller clusters contain fewer than 1,000 papers.

This reflects the heterogeneous nature of academic citation networks, where broad research areas coexist with smaller, specialized topics.
Overall, the results indicate the presence of large, dominant communities, representing broad research domains, alongside smaller communities corresponding sub-fields or specialized topics.

4.1.6 Cluster Composition by Dominant Field of Study Across Time Windows

	Alluvial Label	Time Window	Cluster	Cluster Size	Top 1 FOS	Top 1 %	Top 2 FOS	Top 2 %	Top 3 FOS	Top 3 %
0	T0_C0	2000-2005	0	962	Computer Science	1.000	Artificial Intelligence	0.000	Data Science	0.000
1	T0_C1	2000-2005	1	286	Computer Science	0.955	Artificial Intelligence	0.045	Data Science	0.000
2	T0_C2	2000-2005	2	69	Robotics	1.000	Computer Science	0.000	Artificial Intelligence	0.000
3	T0_C3	2000-2005	3	310	Data Science	0.800	Physics	0.139	Machine Learning	0.061
4	T0_C4	2000-2005	4	148	Mathematics	0.858	Computer Science	0.135	Artificial Intelligence	0.007
5	T0_C5	2000-2005	5	103	Data Science	0.874	Physics	0.078	Computer Science	0.029
6	T0_C6	2000-2005	6	37	Artificial Intelligence	1.000	Computer Science	0.000	Data Science	0.000
7	T0_C7	2000-2005	7	31	Robotics	0.581	Computer Science	0.290	Artificial Intelligence	0.065
8	T0_C8	2000-2005	8	484	Mathematics	1.000	Computer Science	0.000	Artificial Intelligence	0.000
9	T0_C9	2000-2005	9	36	Mathematics	0.778	Robotics	0.222	Artificial Intelligence	0.000
10	T1_C0	2000-2010	0	6412	Computer Science	0.998	Computational Science	0.001	Physics	0.001
11	T1_C1	2000-2010	1	8380	Computational Science	0.993	Artificial Intelligence	0.007	Computer Science	0.000
12	T1_C2	2000-2010	2	1294	Computer Science	0.995	Mathematics	0.005	Computational Science	0.000
13	T1_C3	2000-2010	3	784	Computer Science	0.865	Mathematics	0.083	Artificial Intelligence	0.026
14	T1_C4	2000-2010	4	2654	Physics	0.931	Computational Science	0.045	Mathematics	0.011
15	T1_C5	2000-2010	5	297	Computer Science	0.805	Physics	0.108	Computational Science	0.057
16	T1_C6	2000-2010	6	411	Data Science	0.616	Computational Science	0.178	Robotics	0.148
17	T1_C7	2000-2010	7	369	Mathematics	0.924	Computational Science	0.054	Robotics	0.014
18	T1_C8	2000-2010	8	593	Data Science	0.737	Robotics	0.219	Machine Learning	0.044
19	T1_C9	2000-2010	9	789	Mathematics	0.915	Physics	0.085	Artificial Intelligence	0.000
20	T2_C0	2000-2015	0	10566	Computational Science	0.805	Computer Science	0.180	Physics	0.008
21	T2_C1	2000-2015	1	2550	Physics	0.987	Data Science	0.011	Computational Science	0.002
22	T2_C2	2000-2015	2	8641	Computer Science	0.903	Computational Science	0.056	Mathematics	0.023
23	T2_C3	2000-2015	3	254	Computer Science	0.760	Computational Science	0.130	Mathematics	0.083
24	T2_C4	2000-2015	4	1055	Mathematics	0.822	Robotics	0.178	Artificial Intelligence	0.000
25	T2_C5	2000-2015	5	737	Data Science	0.962	Machine Learning	0.038	Computational Science	0.000
26	T2_C6	2000-2015	6	871	Mathematics	0.783	Robotics	0.148	Computer Science	0.042
27	T2_C7	2000-2015	7	282	Computer Science	0.610	Computational Science	0.248	Data Science	0.053
28	T2_C8	2000-2015	8	546	Data Science	0.967	Machine Learning	0.033	Computational Science	0.000
29	T3_C0	2000-2020	0	8817	Computational Science	0.986	Computer Science	0.014	Artificial Intelligence	0.000
30	T3_C1	2000-2020	1	6747	Computer Science	0.954	Computational Science	0.041	Data Science	0.004
31	T3_C2	2000-2020	2	1789	Mathematics	0.941	Computer Science	0.042	Machine Learning	0.017
32	T3_C3	2000-2020	3	452	Data Science	0.887	Computational Science	0.091	Robotics	0.009
33	T3_C4	2000-2020	4	1112	Data Science	0.790	Artificial Intelligence	0.125	Computational Science	0.069
34	T3_C5	2000-2020	5	1358	Computer Science	0.947	Robotics	0.024	Mathematics	0.010
35	T3_C6	2000-2020	6	2188	Computer Science	0.876	Robotics	0.088	Physics	0.037
36	T3_C7	2000-2020	7	205	Robotics	0.605	Artificial Intelligence	0.166	Computer Science	0.161
37	T3_C8	2000-2020	8	290	Computer Science	0.693	Physics	0.159	Mathematics	0.076
38	T3_C9	2000-2020	9	2604	Physics	0.950	Mathematics	0.028	Computer Science	0.017

Table 2: Detailed Cluster Legend - Top 3 FOS per Cluster

Computer Science emerges as the most dominant and persistent field across all time windows, often appearing in multiple clusters.

This suggests that it is not a single homogeneous field, but rather composed of multiple internal sub-communities.

From the 2000–2010 snapshot onward, Computational Science emerges as a large and persistent community, indicating its growing structural importance.

Other fields such as Physics, Mathematics, and Data Science form distinct clusters, while Artificial Intelligence and Machine Learning typically appear as secondary fields within broader domains.

4.1.7 Temporal Community Analysis

Consecutive Snapshots	Average Matched Similarity
2000–2005 → 2000–2010	0.273477
2000–2010 → 2000–2015	0.639871
2000–2015 → 2000–2020	0.614229

Table 3: Average Similarity Between Matched Communities Across Consecutive Snapshots .

To analyze community evolution, clusters in consecutive snapshots were aligned using the Hungarian algorithm and compared using Jaccard similarity.

The results show a significant increase in similarity from 0.273 to 0.640, followed by stabilization and a slight decrease to 0.614.

This indicates a transition from rapid structural changes in early stages to increasing stability in later periods.

4.1.8 Community Evolution Visualization

Alluvial Diagram (Cluster-Level)

To complement the quantitative analysis, an alluvial diagram visualizes community evolution across four temporal snapshots.

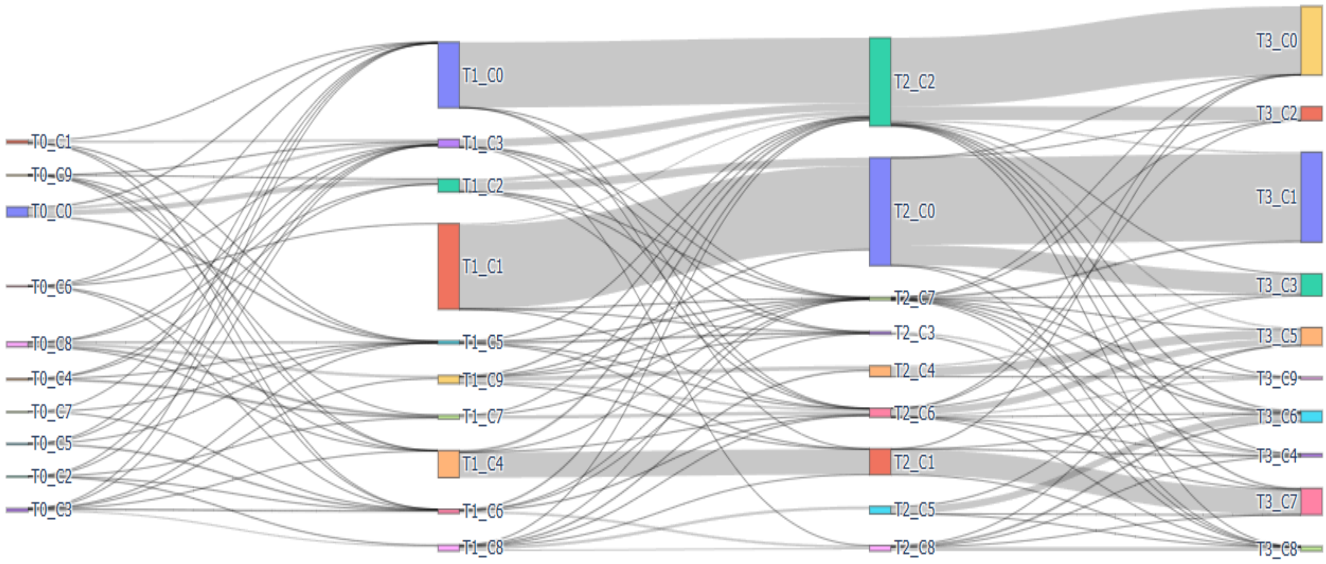

Figure 4: Alluvial Diagram

The Alluvial diagram illustrates how research communities evolve across cumulative time windows, highlighting flows between clusters in consecutive snapshots.

A central pattern is the persistence of large and stable communities over time. Computer Science remains dominant across all snapshots, forming a consistent structural backbone of the network, as reflected by strong and continuous flows.

From the 2000–2010 snapshot onward, Computational Science emerges as a large and persistent community. Its continuity across time indicates its growing structural importance within the citation network.

In contrast, fields such as Data Science, Mathematics, Robotics, and Artificial Intelligence appear as thinner and more fragmented flows, suggesting smaller, more specialized, or dynamically evolving communities.

Window	Clusters
T0: 2000–2005	C0 Computer Science, C1 Computer Science, C2 Robotics, C3 Data Science, C4 Mathematics, C5 Data Science, C6 AI, C7 Robotics, C8 Mathematics, C9 Mathematics
T1: 2000–2010	C0 Computer Science, C1 Computational Science, C2 Computer Science, C3 Computer Science, C4 Physics, C5 Computer Science, C6 Data Science, C7 Mathematics, C8 Data Science, C9 Mathematics
T2: 2000–2015	C0 Computational Science, C1 Physics, C2 Computer Science, C3 Computer Science, C4 Mathematics, C5 Data Science, C6 Mathematics, C7 Computer Science, C8 Data Science
T3: 2000–2020	C0 Computer Science, C1 Computational Science, C2 Computer Science, C3 Computer Science, C4 Computer Science, C5 Mathematics, C6 Data Science, C7 Physics, C8 Data Science, C9 Robotics

Table 4: Cluster legend.

Table 4 provides the cluster legend used in Figure 3, mapping each cluster to its dominant Field of Study (FOS) in each time window.
This mapping enables semantic interpretation of the Alluvial flows by assigning meaningful labels to otherwise abstract clusters.

The table reveals that dominant fields such as Computer Science often appear in multiple clusters within the same snapshot, indicating that they are internally composed of several sub-communities rather than forming a single homogeneous group.

Overall, the Alluvial diagram, supported by Table 4, shows that the network evolution is not uniform, but rather combines structural persistence with dynamic processes such as splitting, merging, and reorganization of communities over time.

Sankey Diagram

To complement the cluster-level view, a FOS-level Sankey diagram presents community evolution based on the dominant Field of Study.

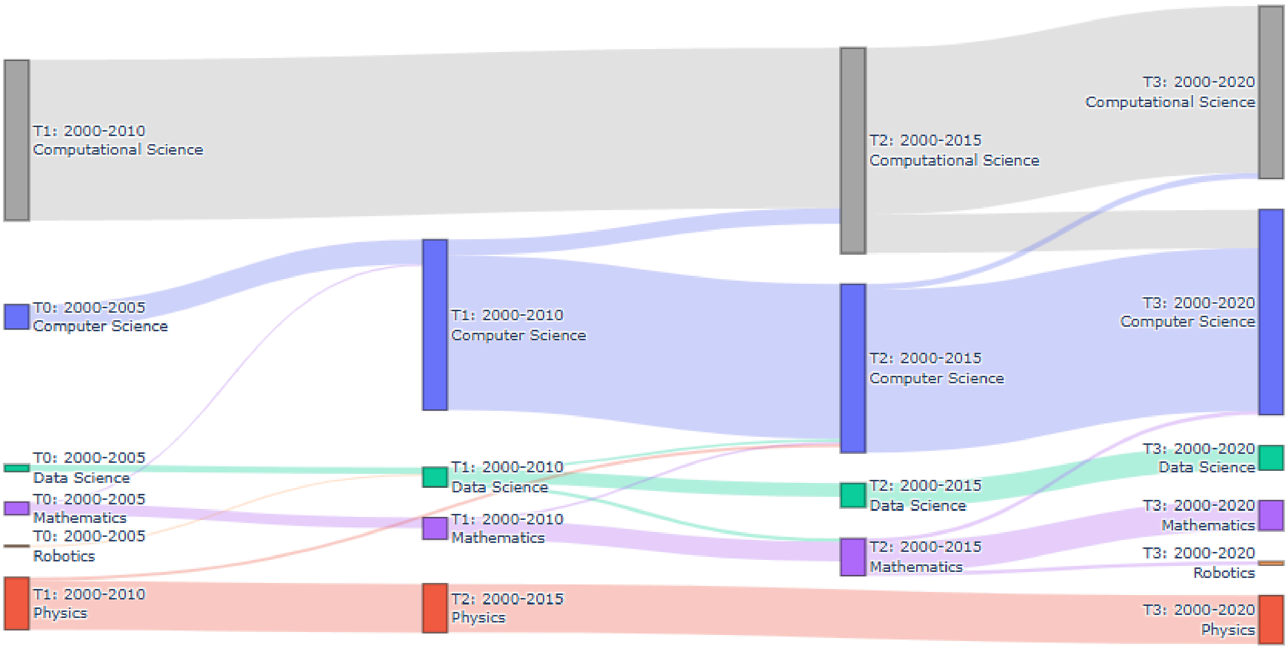

Figure 5: FOS-Level Sankey Diagram

To complement the cluster-level view, the Sankey diagram presents community evolution at the level of dominant Fields of Study.

Each flow represents the transition of dominant research fields across time, with flow width indicating relative volume.

Computer Science maintains a continuous presence across all time windows, reinforcing its role as a core domain in the network.
Computational Science emerges as a dominant and stable field from 2010 onward, aligning with the patterns observed in the Alluvial diagram.

Fields such as Physics show relative stability, while Data Science, Mathematics, Robotics, and Artificial Intelligence form thinner flows, indicating either specialization or integration into broader domains.

Compared to the Alluvial diagram, the Sankey representation provides a clearer semantic perspective, emphasizing how research domains evolve, merge, or persist over time.

Together, the Alluvial and Sankey visualizations reveal a hierarchical structure in the citation network, where dominant research domains provide long-term stability, while smaller domains exhibit dynamic and adaptive behavior.

4.2 Conclusions and Future Work

4.2.1 Conclusions

This project presented a framework for temporal community detection in citation networks by combining graph representation learning, clustering techniques, and temporal alignment methods.

The experimental results demonstrate that the framework is capable of detecting meaningful community structures within large-scale citation networks. Across all temporal snapshots, the clustering process achieved relatively high Silhouette Scores, indicating good separation between communities and strong internal cohesion.

In addition, the robustness analysis showed that the clustering results are generally stable across multiple K-Means runs, supporting the reliability of the proposed methodology.

The temporal analysis showed that the similarity between consecutive snapshots increased after the initial stage and then stabilized, suggesting that research communities become more mature and structurally consistent over time.

The cluster-level Alluvial diagram illustrated the dynamic behavior of communities, including persistence, growth, splitting, and merging. In addition, the FOS-level Sankey diagram provided a clearer semantic interpretation of how dominant research fields evolve over time.

Overall, the proposed framework successfully combines graph-based learning and temporal analysis to provide insights into the evolution of scientific knowledge.

4.2.2 Future Work

Several directions may be explored in future research.

First, additional graph neural network architectures, such as Graph Attention Networks (GAT) or Graph Isomorphism Networks (GIN), could be evaluated to examine whether they provide improved node representations and clustering performance.

Second, richer node features could be incorporated including textual information from paper titles, abstracts, keywords, or author metadata, to enhance semantic representation.

Third, more advanced community detection techniques could be investigated, as graph-specific algorithms may reveal additional structural patterns beyond K-Means.

Another possible extension is the use of non-cumulative temporal or sliding time windows which may provide a more detailed view of short-term changes and emerging research trends.

Finally, the proposed framework could be applied to larger and more diverse citation datasets to evaluate its scalability and generalization across domains.

References

[1] Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive representation learning on large graphs. *arXiv preprint arXiv*. https://arxiv.org/abs/1706.02216

[2] Hubert, L., & Arabie, P. (1985). Comparing partitions. *Journal of Classification, 2*(1), 193–218. https://doi.org/10.1007/BF01908075

[3] Jaccard, P. (1901). Étude comparative de la distribution florale dans une portion des Alpes et des Jura. *Bulletin de la Société Vaudoise des Sciences Naturelles, 37*, 547–579. https://www.biodiversitylibrary.org/page/4612088

[4] Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly, 2*(1–2), 83–97. https://doi.org/10.1002/nav.3800020109

[5] Lloyd, S. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory, 28*(2), 129–137. https://doi.org/10.1109/TIT.1982.1056489

[6] Milligan, G. W., & Cooper, M. C. (1986). A study of the comparability of external criteria for hierarchical cluster analysis. *Multivariate Behavioral Research, 21*(4), 441–458. https://doi.org/10.1207/s15327906mbr2104_5

[7] Newman, M. (2018). *Networks*. Oxford University Press. https://doi.org/10.1093/oso/9780198805090.001.0001

[8] Price, D. J. de S. (1965). Networks of scientific papers. *Science, 149*(3683), 510–515. https://doi.org/10.1126/science.149.3683.510

[9] Rosvall, M., & Bergstrom, C. T. (2010). Mapping change in large networks. *PLoS ONE, 5*(1), e8694. https://doi.org/10.1371/journal.pone.0008694

[10] Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics, 20*, 53–65. https://doi.org/10.1016/0377-0427(87)90125-7

[11] PyTorch Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. Advances in Neural Information Processing Systems (NeurIPS), 32.

https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library

[12] PyTorch Geometric Fey, M., & Lenssen, J. E. (2019). Fast Graph Representation Learning with PyTorch Geometric. ICLR Workshop on Representation Learning on Graphs and Manifolds. https://arxiv.org/abs/1903.02428

[13] Scikit-learn Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

[14] Plotly Plotly Technologies Inc. (2024). Plotly Python Open Source Graphing Library. https://plotly.com/python/

Appendix

Appendix A – User Guide

Purpose

This system enables the analysis of temporal community evolution in citation networks. The user can generate citation graph snapshots, detect research communities, compare communities across time windows, and visualize their evolution.

System Requirements

- Python 3.x
- Google Colab environment
- Google Drive account
- Required libraries:
    - PyTorch
    - PyTorch Geometric
    - Pandas
    - NumPy
    - Scikit-learn
    - SciPy
    - Plotly
    - Matplotlib

Input Data

The system requires two main data sources:

1. Papers dataset:

    - paper_id
    - publication year
    - field of study
2. Citation dataset:
    - source paper
    - target paper

The data is loaded automatically through the data loading modules.

Running the System

Step 1 – Open the Notebook

Open the project notebook in Google Colab and mount Google Drive.

Step 2 – Install Dependencies

Run the initialization cells to install required libraries and import all project modules.

Step 3 – Configure Time Windows

Define the desired cumulative time windows:

```
time_windows = [
    (2000, 2005),
    (2000, 2010),
    (2000, 2015),
    (2000, 2020),
]
```

Step 4 – Execute the Pipeline

Run the final execution section.

The system automatically performs the following pipeline stages:

- Data loading
- Graph construction
- Feature generation
- GraphSAGE embedding learning
- Community detection (clustering stage)
- Robustness evaluation
- Temporal community alignment (alignment stage)
- Alluvial visualization (visualization stage)

Outputs

The system produces:

- Number of papers and edges per snapshot
- Graph embeddings
- Community assignments
- Optimal number of clusters
- Silhouette scores
- Robustness scores
- Temporal similarity measures (used in the alignment stage)
- Alluvial diagram visualization

Interpretation of Results

Higher Silhouette Scores indicate better cluster separation.

Higher Robustness values indicate more stable clustering results.

Higher temporal similarity values indicate stronger persistence of communities between consecutive time windows.

Appendix B – Maintenance Guide

System Architecture

The project is organized into modular components located in the External_Models directory, following the same pipeline structure described in Chapter 3.

Data Layer (Data Processing Stage)

Responsible for:

- Loading raw data
- Cleaning citation relations
- Creating temporal snapshots
- Building node features
- Converting data to PyTorch Geometric format

Modules:

- loader.py
- graph_builder.py
- snapshot_builder.py
- feature_builder.py
- pyg_converter.py

Embedding Layer (Embedding Stage)

Responsible for GraphSAGE representation learning.

Modules:

- graphsage_model.py
- embedder.py

Clustering Layer (Community Detection Stage)

Responsible for:

- K-Means clustering
- Selection of optimal K
- Robustness analysis

Modules:

- kmeans_optimizer.py
- robustness.py

Alignment Layer (Temporal Alignment Stage)

Responsible for temporal community matching.

Modules:

- similarity_metrics.py
- hungarian_matcher.py
- temporal_matcher.py

Visualization Layer (Visualization Stage)

Responsible for generating alluvial diagrams.

Module:

- alluvial_plotter.py

Extending the System

The system can be extended in several ways:

- Replace GraphSAGE with alternative GNN architectures such as GCN or GAT.
- Introduce additional node features.

- Apply alternative clustering algorithms.
- Add new temporal similarity measures.
- Support non-cumulative temporal windows.

Updating the Dataset

To use a new dataset:

1. Replace the raw input files.
2. Verify that required fields exist.
3. Run the data loading and graph construction stages.
4. Execute the pipeline normally.

No additional code modifications are required, as long as the data schema remains unchanged and is consistent with the data processing stage.

Troubleshooting

Import Errors

Verify that the External_Models directory is included in the Python path.

Empty Snapshots

Check that papers exist within the selected time window.

Clustering Errors

Verify that the embedding matrix contains enough nodes for the selected K range.

Alignment Errors

Verify that temporal similarity matrices are correctly computed before the temporal alignment stage.

Visualization Issues

Ensure that Plotly is installed and that the browser supports interactive HTML visualizations.

Future Maintenance

Future versions may include:

- Automated hyperparameter tuning
- Dynamic graph neural networks

- Real-time citation updates
- Advanced temporal community tracking
- Interactive web-based dashboards